Supervisor Meeting

1. Original Results

TL;DR

Two blocks of training experiments on **Qwen3-0.6B** (Apr 3 to Apr 19, all on 1× A100 on the ALICE cluster) tested whether a small tool-using LM can be GRPO-trained on MuSiQue with the Search-R1 paper's reward. Findings: **the hybrid model learns slowly but stably; the base model cannot learn the tool-call format from cold-start; prompt design dominates behavior; the paper's partial-credit reward creates a 0.1 floor that masks the tool-use signal**. Training on Qwen3-0.6B was concluded as too slow to support reward-function ablation in the remaining timeline. We then reproduced the **Search-R1 evaluation baseline** on `Qwen2.5-3B-Search-R1` (paper checkpoints, 7 QA datasets, 1k subsamples, ±2.5 pp of paper) and pivoted training to **Qwen3.5-2B on NeMo-RL** (verl does not support Qwen3.5).

Storyline plot

Five runs that anchor the story, all Qwen3-0.6B on 1× A100 ALICE. **v0 best** (`p3_decide_no_ex` , blue solid) and **v0 collapse** (`p1_basic_no_ex` , orange dashed) bracket the prompt-sensitivity range; both have rewards within 0.16 to 0.22 due to the 0.1 partial-credit floor. **v1 best** (`r1_query_object` , green solid) ports the same regime to the in-distribution `<tool_call>` tag format and reaches similar reward at equal step count. **v1 base attempt** (`base_state_machine_a` , red dashed) ran 2300 steps with 0 tool calls throughout; the base model cannot produce the structured format from cold-start.

What was tried

Block	Period	Runs	What was tested
v0: Qwen3-0.6B, paper `<search>` / `<result>` tags	Apr 3 to Apr 9	14	Base vs hybrid (instruct); 9-prompt ablation on hybrid (varying rules verbosity, with vs without few-shot example)
v1: Qwen3-0.6B, in-distribution `<tool_call>` tags	Apr 12 to Apr 19	15	New prompt format (instruct, 3 prompts); 5 fresh base-model attempts; some reward-function probing

Full details and per-run plots have been recorded.

What I learned

1. **Base model cannot follow the tool-use format from cold start.** Across 5 v1 base-model runs (longest 2300 steps), `tool_call_counts/mean` stayed at 0 throughout. Without instruction tuning or SFT warm-start, GRPO does not bootstrap the structured format on a 0.6B base. (One v1 base run shows reward 0.7 due to a reward-function code change mid-block; ignored.) Decision: focus on the hybrid checkpoint.
2. **Hybrid does learn, slowly.** All 9 prompt-ablation runs in v0 show monotonic reward gains; final means cluster at 0.18 to 0.22 over up to 2300 steps. Training is numerically stable on 1× A100-40GB at the constrained sequence budget (`max_response_length=4096` , `rollout.n=3`).
3. **Prompt drives behavior more than reward.** With training config and reward held constant, prompt phrasing alone moved end-of-run tool-call rate from 0 to 2 per episode, response length from ~480 to ~2050 tokens. The reward only moved ±3 pp across the same prompts. Specifically: removing the few-shot example collapsed tool use to 0 unless the rules section had explicit per-step decision guidance; then tool use survived removal and gave the best reward of v0 (0.215).

4. **The paper's partial-credit reward creates a floor that masks the signal.** The reward returns 0.1 for any well-formatted but wrong answer. Even runs that abandoned tool use entirely finished at 0.16 reward; tool-using runs at 0.18 to 0.22. The 3 to 6 pp gap between tool-using and no-tool behaviors is too small to clearly drive learning. **Conditioning the partial credit on a correct answer (or removing it entirely)** is a planned future ablation.
5. **The `<tool_call>` in-distribution format costs nothing at equal step count.** v1's best instruct prompt (`r1_query_object` , 884 steps) reached reward 0.179, matching v0's mid-pack at the same step count. v0's higher peak (0.215) just reflects 2.6× more training steps, not a better tag format.
6. **Training Qwen3-0.6B is too slow for reward-function ablation under the timeline.** Even with the constrained config (sequences halved from 8192 to 4096; rollout width 5 → 3), the full 9968-step horizon is multiple days per run. Multiple reward-function variants × seeds is not feasible at this rate on 1× A100.

Search-R1 evaluation baseline (reproduction)

To anchor downstream comparisons, the published `Qwen2.5-3B-Search-R1` checkpoints were re-evaluated on the paper's 7 QA datasets. There was no eval pipeline provided; one was built from scratch, then locked after a 3-fix audit (apply_chat for base, restored example sentence, removed runtime add_special_tokens block).

Subsampling: full eval sets total ~50K rows. We ran **1000-row stratified subsamples** for the six large datasets (NQ, TriviaQA, PopQA, HotpotQA, 2Wiki, plus Bamboogle full @ 125 rows and MuSiQue full @ 2417 rows), single seed, greedy decode.

Results (avg across 7 datasets):

Variant	Ours (avg EM)	Paper (avg EM)	Δ
base	**0.292**	0.312	–2.0 pp
instruct	**0.361**	0.336	+2.5 pp

Both within ±5 pp of paper on 6 of 7 datasets; format-validity (`</answer>` close-rate) ≥99.6 % on base and ≥91.4 % on instruct.

Original Phase-2 plan (not feasible at full scope)

1. Re-run Search-R1 eval on **full benchmarks × 5 seeds** (not 1k subsamples). Forced an index switch: Flat IP (65 GB, paper-fidelity) → **IVF-SQ8** (16 GB, 3 to 10× faster); Flat IP times out under

training rollout load.

2. Evaluate **untrained Qwen3.5-2B** (base + hybrid) for the no-training floor.
3. **Train Qwen3.5-2B on NeMo-RL** (base + hybrid). Pipeline ported and launch-ready.
4. **Ablate the reward function** (EM-only vs. shaped vs. no-partial-credit) per the v0 floor finding.

Compute reality kills (3) at scale and (4) entirely. (1) and (2) are still cheap.

Proposed reframing

Original RQs (Plan B): RQ1 domain expansion approaches, RQ2 reward function modeling, RQ3 meta-reasoning, RQ4 curriculum-based training.

Pivot to:

> **Is it feasible to post-train a small LM to Search-R1-level results under realistic resource constraints, and what is the optimised training recipe?**

Uses the artifacts already produced (prompt-sensitivity, partial-credit floor, reproduced eval baseline, ported NeMo-RL pipeline) as primary evidence; answerable with one or two trained runs rather than a sweep.

Pivot to Qwen3.5-2B on NeMo-RL

After the Qwen3-0.6B compute reality became clear, training pivoted to **Qwen3.5-2B** (closer to paper's Qwen2.5-3B in capacity, and Qwen3.5 has the same hybrid soft-switch reasoning toggle). Two blockers:

- **Framework**: `verl` does not support Qwen3.5. The Search-R1-style pipeline was ported to **NeMo-RL** (NVIDIA's GRPO/RLVR framework, Qwen3.5-supported). Done as an overlay under `training/src/` : custom Ray-actor environment for the retriever, byte-identical port of the paper's reward function (15 parity tests pass), unchanged paper hyperparameters.
- **Wall-clock (observed from Vast.ai 1× A100 80GB smoke)**: ~57 s/step for 20 trajectories (smoke shape). Scaled to the paper batch shape (510 trajectories/step × 1005 steps), this projects to **~11 to 17 days per run on 1× A100 80GB** (~$300 to $490 / run on Vast). On 1× H100 80GB SXM: ~5 to 8.5 days, $240 to $410 / run. On 2× A100: ~6.5 to 9.5 days, $370 to $550 / run.
- **Reward-function caveat surfaced during smoke**: Search-R1's GitHub ships two reward functions (`qa_em.py` paper-faithful EM-only, `qa_em_format.py` 6-tier shaped with non-zero defaults). Earlier docs in this project conflated them; the shaped variant produces visible partial-

credit reward even when EM=0. The Phase-2 NeMo-RL port now uses the EM-only paper-faithful version.

Net conclusion (where this leaves us)

- The hybrid Qwen3-0.6B work demonstrates the **stability** of the GRPO + multi-turn tool-call pipeline on 1 A100 and **identifies the partial-credit reward floor** as the most actionable methodological lever.
- The **base model is a dead end** on a 0.6B without SFT warm-start; not worth retrying at the 2B size unless an SFT or distillation step is added.
- **Reward-function ablation at Qwen3.5-2B scale is not feasible** within the remaining compute budget. Observed wall-clock is **11 to 17 days per run on 1× A100 80GB**.

2. Next Steps: Proposed Training Recipe

Three drop-in additions to a single Search-R1 GRPO baseline run on Qwen3.5-2B, all chosen because they target the "1× A100 budget" reframing and are cheap to add:

Technique	What it is	Proven result	Cost to add
E2H curriculum (data-level) (2506.06632)	Train NQ (1-hop) → HotpotQA (2-hop) → MuSiQue (3-hop), ~300 steps per stage; fade out earlier stages	Vanilla GRPO fails on hard tasks for 1.5-3B LLMs; curriculum recovers them across multiple domains	Data scheduler only; no GRPO code change
S-GRPO (2504.20834)	Compute the policy-gradient loss on 30-50 % of tokens per rollout (informativeness-sampled)	Qwen2-1.5B + LoRA: vanilla GRPO 0 gain → S-GRPO **+24 pp** on SVAMP. Roughly 2× backprop saving per step	One change in the loss path
MC-GRPO (2601.22582)	Replace mean baseline with group **median** (G+1 rollouts, exclude pivot)	Closes the G=2 to G=8 quality gap to within 1 % across model families	One-line baseline swap

Combined, these are the candidate answer to "what's the optimised training recipe?". They stack on a single GRPO baseline run, fit on 1× A100, and are testable within 2 to 3 affordable runs.

Required control: per JustRL (2512.16649), curriculum / length-penalty / dynamic-sampling stacks can *degrade* OOD performance by collapsing exploration. **Run a plain-GRPO control alongside the recipe**; if minimal beats the stack, the stack is hurting.